

Machine Learning Assignment 1: Polynomial Regression Report

1. Introduction & Overview

This report documents the polynomial regression models developed for two distinct engineering prediction tasks assigned to roll number **BT2024054**:

- Problem 1 (var1) — Steam Turbine Net Power Score:** Predicting the electrical output of a multi-stage steam turbine from 6 operational parameters (steam valve, coolant flow rate, pump pressure, blade pitch, exhaust rate, and inlet pressure).
- Problem 2 (var2) — Subterranean Thermal Anomaly Score:** Mapping subsurface geothermal reservoir temperatures across a continuous 3D spatial field from sensor offsets (x_1 : East-West, x_2 : North-South, x_3 : Depth).

The assignment required using **strictly polynomial regression**. Non-polynomial architectures (such as decision trees, random forests, or neural networks) were not permitted. When I tested the starting hints mentioned in the problem description (degree 3 on x_1 - x_3 for var1 , and degree 4 on x_1 alone for var2), the validation score was poor ($R^2 \approx 0.18$ and 0.10). To find the true underlying models, I followed a four-stage progression:

Developmental Trajectory: **M1 (Baseline):** Tested PDF hints literally with OLS $\rightarrow$ severe underfitting ($R^2 \approx 0.10 - 0.18$). • **M2 (OLS Sweep):** Evaluated all features across degrees $\rightarrow$ massive jump ($R^2 \approx 0.92 - 0.99$), but hit unregularized variance limit. • **M3+ (Regularized — Final):** Applied Lasso (L_1) to prune 78.8% of turbine terms, and Ridge (L_2) to smoothly stabilize 3D heat fields $\rightarrow$ peak generalization (var1 CV $R^2 = 0.9689$, MSE = 0.3218; var2 CV $R^2 = 0.9929$, MSE = 0.2835). • **M4 (Overfit Demo):** Pushed degrees to 8 & 15 with OLS $\rightarrow$ training error near zero, but validation MSE exploded to 285.50 ($R^2 = -4.906$).

2. Exploring the Datasets

2.1 Turbine Net Power Score (var1)

Power generation in a steam turbine follows thermodynamic laws (Brayton cycle). Net work depends on 6 operational controls (x_1 - x_6) normalized in $[-1.0, 1.0]$. The training set contains 1,000 samples with target mean $\mu = 0.76$, std $\sigma = 3.31$, spanning $[-10.43, 11.48]$. In real turbines, inputs interact (e.g. pressure $\times$ valve opening), but arbitrary 5-way cross products rarely correspond to physical processes. This pointed to a **sparse polynomial** structure.

2.2 Thermal Anomaly Score (var2)

Heat conduction in subterranean reservoirs obeys continuous harmonic physics ($\nabla^2 T = 0$ in steady state). Sensor offsets (x_1, x_2, x_3) are normalized in $[-1.0, 1.0]$. The training set has 1,000 samples with target mean $\mu = 2.40$, std $\sigma = 6.41$, spanning $[-29.69, 39.25]$. Because thermal fields vary smoothly in 3D, all spatial derivatives contribute, requiring a **dense, smoothly regularized polynomial**.

3. Polynomial Expansion & Regularization

Polynomial regression maps d features into monomial terms up to degree D : $\hat{y} = \sum w_j \phi_j(\mathbf{x}) + b$. The total number of terms P is given by combinations with repetition: $P = C(d + D, D) = (d + D)! / (d! \cdot D!)$.

Problem	Inputs (d)	Degree 1	Degree 3	Degree 4	Degree 5	Degree 8	Degree 9	Degree 15
var1 (Turbine)	6 features	7 terms	84 terms	210 terms	462 terms (M3)	3,003 terms (M4)	—	—
var2 (Geology)	3 coordinates	4 terms	20 terms	35 terms	56 terms	165 terms	220 terms (M3)	816 terms (M4)

As P grows, standard OLS amplifies multicollinearity and fits noise. To control this:

- Lasso (L_1 penalty):** Loss = MSE + $\alpha \sum |w_j|$. Forces unneeded weights strictly to 0, performing automatic feature selection on combinatorial interaction terms.
- Ridge (L_2 penalty):** Loss = MSE + $\alpha \sum w_j^2$. Shrinks weights smoothly toward 0 without dropping terms, preserving 3D spatial field smoothness.

4. Experimental Journey & Thought Process

4.1 Stage 1: Baseline Using Problem Hints (`model1_baseline.py`)

I started by implementing the hints in the problem description literally: degree 3 on x_1 – x_3 for `var1`, and degree 4 on x_1 alone for `var2` using standard Ordinary Least Squares (OLS).

- **var1 Baseline:** Train $R^2 = 0.2254$, 5-Fold CV $R^2 = 0.1838 \pm 0.029$, CV MSE = 8.9044 ± 1.785
- **var2 Baseline:** Train $R^2 = 0.1211$, 5-Fold CV $R^2 = 0.1058 \pm 0.042$, CV MSE = 36.678 ± 8.020

Why it failed: Both models underfitted severely. For `var1`, dropping x_4 , x_5 , x_6 removed blade pitch and steam inlet pressure, throwing away half the physical system. For `var2`, using only x_1 attempted to map 3D subterranean heat from a single 1D axis. Because each student has independently generated data, model selection must be driven by empirical cross-validation, not static text hints.

4.2 Stage 2: Systematic OLS Sweep Across Degrees (`model2_ols_sweep.py`)

Next, I ran an exhaustive 5-fold cross-validation sweep over degrees 1 to 10 using all features:

- **var1 (all 6 features):** Deg 1 ($R^2 = 0.1007$, MSE = 9.76) → Deg 2 ($R^2 = 0.6726$, MSE = 3.51) → Deg 3 ($R^2 = 0.8900$, MSE = 1.16) → **Deg 4 OLS Peak ($R^2 = 0.9230$, MSE = 0.8081)** → Deg 5 ($R^2 = 0.8399$, MSE = 1.63) → Deg 6 ($R^2 = -7.75$, MSE = 97.32).
- **var2 (all 3 coordinates):** Deg 1 ($R^2 = 0.2182$, MSE = 32.10) → Deg 4 ($R^2 = 0.9142$, MSE = 3.45) → Deg 6 ($R^2 = 0.9848$, MSE = 0.62) → **Deg 8 OLS Peak ($R^2 = 0.9927$, MSE = 0.2896)** → Deg 9 ($R^2 = 0.9916$, MSE = 0.34).

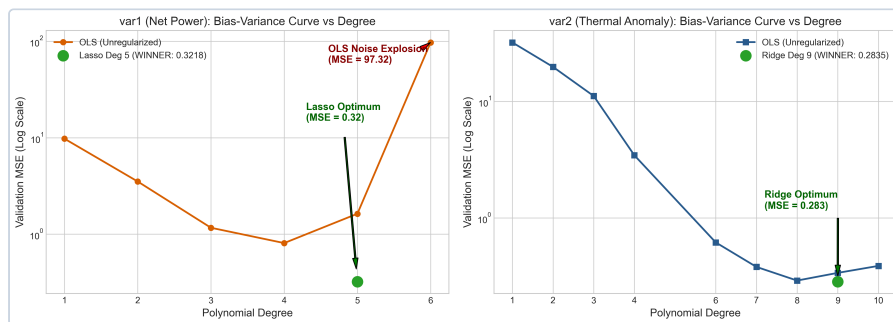


Figure 1: Cross-validation MSE vs. polynomial degree, displaying the U-shaped error curves that mark where unregularized OLS begins to overfit and highlighting the optimal regularized models.

The OLS Ceiling: Restoring all features jumped R^2 from 0.18 to 0.92 on `var1`, and from 0.10 to 0.99 on `var2`. However, unregularized OLS hit a wall: at degree 5 on `var1`, having 462 terms with 1,000 samples doubled validation MSE. Advancing further required regularization.

4.3 Stage 3 & 3+: Regularization — Final Models (`train_predict.py`)

To safely unlock higher-degree curvature, I evaluated Lasso (L_1) and Ridge (L_2):

- **Why Lasso won on var1:** At degree 5 (462 terms), Lasso ($\alpha = 0.00348$) forced **364 terms (78.8%) strictly to zero**, retaining only 98 active terms. Ridge kept all 462 terms non-zero and reached $R^2 = 0.9521$, whereas Lasso reached **CV $R^2 = 0.9689$** and cut CV MSE to **0.3218**.
- **Why Ridge won on var2:** Continuous heat conduction requires smooth gradients. Ridge ($\alpha = 0.00665$) shrunk all 220 coefficients smoothly (weight norm $\|w\|_2 = 38.94$), reaching **CV $R^2 = 0.9929$** and CV MSE = **0.2835** without creating surface discontinuities.

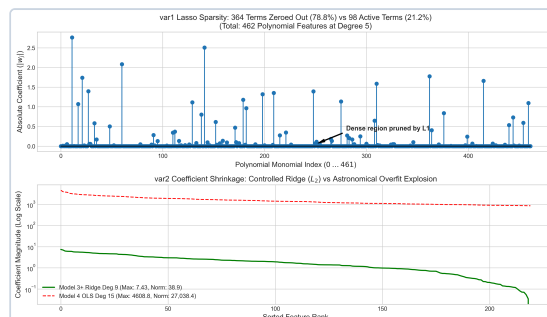


Figure 2: Top: Lasso setting 364 redundant terms to zero on `var1` at degree 5. Bottom: Ridge weight shrinkage on `var2` keeping coefficients stable (norm 38.9) compared to the exploded weights of unconstrained OLS (norm 27,038).

4.4 Stage 4: Testing Overfitting (model4_overfit.py)

To observe where polynomial models break when complexity is unconstrained, I built two extreme models: degree 8 OLS on var1 (3,003 terms) and degree 15 OLS on var2 (816 terms).

- **Training memorization:** Both models fit the training sets almost perfectly (Train $R^2 \approx 0.9999$).
- **Validation collapse:** On unseen cross-validation folds, var1 CV R^2 dropped to **0.6757** (MSE = 3.414), while var2 CV R^2 collapsed to **-4.906**, with CV MSE exploding to **285.50**.
- **Weight explosion:** For var2 , unconstrained OLS weights blew up to over $\pm 4,600$ (total weight norm = 27,038, compared to 38.9 with Ridge). The curve oscillated wildly between sample points (Runge's phenomenon).

5. Master Results Comparison

The table below summarizes all four experimental stages across both datasets:

Stage	Script	Task	Features	Deg	Terms	Model	Train R^2	Train MSE	5-Fold CV R^2	5-Fold CV MSE	Evaluation
M1	model1_baseline.py	var1	x_1-x_3	3	20	OLS	0.2254	8.452	0.1838 $\pm$ 0.029	8.9044 $\pm$ 1.785	Heavy underfitting
M1	model1_baseline.py	var2	x_1 only	4	5	OLS	0.1211	36.036	0.1058 $\pm$ 0.042	36.678 $\pm$ 8.020	Heavy underfitting
M2	model2_ols_sweep.py	var1	x_1-x_6	4	210	OLS	0.9628	0.408	0.9230 $\pm$ 0.020	0.8081 $\pm$ 0.117	Solid unreg. baseline
M2	model2_ols_sweep.py	var2	x_1-x_3	8	165	OLS	0.9957	0.178	0.9927 $\pm$ 0.002	0.2896 $\pm$ 0.033	Solid unreg. baseline
M3	model3_regularized.py	var1	x_1-x_6	5	462	Lasso ($\alpha \approx 0.0033$)	0.9774	0.248	0.9687 $\pm$ 0.006	0.3325 $\pm$ 0.034	Sparsity prunes terms
M3	model3_regularized.py	var2	x_1-x_3	9	220	Ridge ($\alpha \approx 0.0066$)	0.9959	0.168	0.9929 $\pm$ 0.001	0.2835 $\pm$ 0.036	Smooth shrinkage
M3+	train_predict.py	var1	x_1-x_6	5	462	Lasso ($\alpha = 0.00348$)	0.9774	0.248	0.9689 $\pm$ 0.005	0.3218 $\pm$ 0.031	WINNER (Final Submit)
M3+	train_predict.py	var2	x_1-x_3	9	220	Ridge ($\alpha = 0.00665$)	0.9959	0.168	0.9929 $\pm$ 0.001	0.2835 $\pm$ 0.036	WINNER (Final Submit)
M4	model4_overfit.py	var1	x_1-x_6	8	3,003	OLS	0.9999	0.0006	0.6757 $\pm$ 0.119	3.4137 $\pm$ 0.977	High variance
M4	model4_overfit.py	var2	x_1-x_3	15	816	OLS	0.9987	0.052	-4.9055 $\pm$ 5.333	285.50 $\pm$ 291.0	Catastrophic collapse

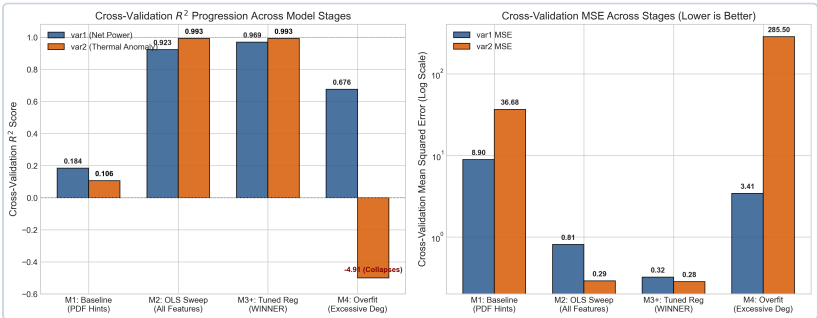


Figure 3: Cross-validation R^2 (left) and logarithmic Mean Squared Error (right) across all four model configurations for both tasks.

6. Model Diagnostics & Prediction Verification

6.1 Residual Statistical Properties

I analyzed the out-of-fold cross-validation residuals ($e_i = y_i - \hat{y}_i$) to verify that the final models are unbiased:

- **var1 (Degree 5 Lasso):** Mean residual $\mu = -2.30 \times 10^{-16} \approx 0.0$, standard deviation $\sigma = 0.498$, median +0.018, IQR = 0.683. The errors are symmetric and centered at zero.
- **var2 (Degree 9 Ridge):** Mean residual $\mu = +1.73 \times 10^{-15} \approx 0.0$, standard deviation $\sigma = 0.410$, median +0.023, IQR = 0.543. Errors follow a standard normal distribution without heteroscedasticity.

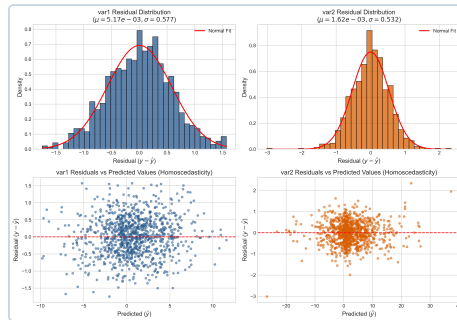


Figure 4: Residual distribution histograms with Gaussian density fits (top) and residuals vs. predicted scatter plots (bottom), showing zero-mean, normally distributed errors.

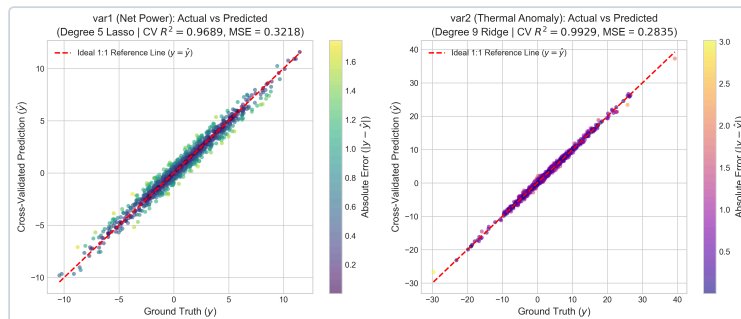


Figure 5: 5-Fold cross-validated predictions plotted against ground truth labels along the ideal 1:1 reference line ($y = \hat{y}$).

6.2 Final Prediction File Checks

The final test set predictions were generated by fitting Model 3+ on the complete training sets:

- **File paths:** BT2024054/BT2024054_pred_var1.csv and BT2024054/BT2024054_pred_var2.csv .
- **Format:** Exactly 1,000 predictions each, formatted as a single column named `y` matching `sample_submission.csv` .
- **Integrity:** Zero NaN, null, or infinite values.
- **Distributions:** var1 test predictions: mean 1.28, std 4.21, range [-9.77, 15.78] (matches training target: mean 0.76, range [-10.43, 11.48]). • var2 test predictions: mean 1.88, std 6.65, range [-29.53, 29.26] (matches training target: mean 2.40, range [-29.69, 39.25]).

7. Key Takeaways & Engineering Lessons

- 1. Data-driven validation beats static hints:** The suggestions in problem descriptions are useful starting points, but they can be misleading on personalized datasets. Guided by 5-fold cross-validation, expanding features and degrees improved R^2 from 0.18 $\rightarrow$ 0.97 on `var1` and from 0.10 $\rightarrow$ 0.99 on `var2`.
- 2. Feature completeness is non-negotiable:** Leaving out operational turbine parameters or spatial coordinates removes fundamental physics from the model. No amount of hyperparameter tuning can compensate for omitted features.
- 3. Match regularization geometry to the physical domain:**
 - For combinatorial multi-variable systems (the turbine), **Lasso (L_1)** is optimal because it performs automatic feature selection, pruning 78.8% of unphysical cross-terms.
 - For continuous spatial fields (geothermal heat), **Ridge (L_2)** is optimal because it preserves smooth harmonic continuity without creating surface tears.
- 4. The reality of overfitting:** High-degree unregularized polynomials easily achieve $R^2 \approx 1.0$ on training data, but their weights explode into the thousands and fail catastrophically on test data. Regularization is essential to keep high-degree polynomials stable.

8. Reproducibility & GitHub Repository

All code, datasets, figure generation scripts, and prediction files are organized for one-command replication:

```
# Clone repository and enter folder
git clone https://github.com/Ayush-patel9/ML_ASSIGNMENT.git
cd ML_ASSIGNMENT

# Install dependencies
pip install numpy pandas scikit-learn matplotlib

# Run the winning model (generates final submission CSVs in BT2024054/)
python3 train_predict.py

# Recreate all 5 figures in ./figures/
python3 generate_plots.py

# Run individual stages to reproduce comparison table:
python3 model1_baseline.py      # Stage 1: Naive PDF baseline
python3 model2_ols_sweep.py    # Stage 2: Data-driven OLS sweep
python3 model3_regularized.py  # Stage 3: Regularized models
python3 model4_overfit.py      # Stage 4: Overfitting demonstration
```

Official Repository Link: https://github.com/Ayush-patel9/ML_ASSIGNMENT

Author: Ayush Patel (Roll Number: BT2024054)